

JDBC Fundamentals and CRUD Operations

A first-principles guide to connecting a Java application to MySQL and performing CRUD operations using JDBC.

Table of Contents

1. [How Java Communicates with a Database](#)
 2. [JDBC API and JDBC Driver](#)
 3. [Installing MySQL Server and DBeaver](#)
 4. [Creating the JDBC Project](#)
 5. [Creating the Database and Table](#)
 6. [Establishing a Database Connection](#)
 7. [Executing SQL with JDBC](#)
 8. [Understanding `ResultSet`](#)
 9. [Why `PreparedStatement` Is Preferred](#)
 10. [Complete CRUD Operations](#)
 11. [Closing JDBC Resources](#)
 12. [Common Setup Problems](#)
 13. [Key Takeaways](#)
-

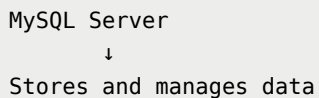
1. How Java Communicates with a Database

Java and MySQL are separate programs

A Java application normally runs inside the JVM:



MySQL runs as a separate server process:



The two programs may run:

- On the same computer
- On different computers in the same network
- On remote servers in different locations

Calling a database is therefore different from calling a normal Java method.

Consider the following code:

```
String sql = "SELECT * FROM students";
```

This only creates a Java `String` in JVM memory. The SQL has not yet reached MySQL.

The application still needs a mechanism that can:

1. Establish a connection with MySQL
2. Send SQL to the server
3. Wait for the server response
4. Convert the response into objects Java can work with

JDBC provides the standard Java-side API for this communication.

Why a database driver is required

Every database server follows its own communication protocol.

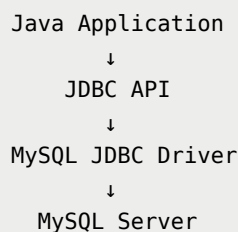
For example:

- MySQL uses the MySQL client-server protocol
- PostgreSQL uses the PostgreSQL protocol
- Oracle Database uses its own protocol

These protocols define:

- How a client opens a connection
- How authentication data is transmitted
- How SQL commands are packaged
- How SQL data types are represented
- How rows and metadata are returned
- How errors are reported
- How transactions are controlled

Java does not contain the implementation of every database protocol. It uses a database-specific JDBC driver as the translator.



2. JDBC API and JDBC Driver

What is JDBC?

JDBC stands for **Java Database Connectivity**.

It is a standard Java API used to communicate with relational databases.

The main JDBC types are available in the `java.sql` package:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
```

JDBC gives Java developers a common programming model for databases such as MySQL, PostgreSQL, Oracle and Microsoft SQL Server.

The database URL and driver may change, but the JDBC interfaces remain largely the same.

MySQL connection

```
Connection connection = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/student_db",
    "root",
    "password"
);
```

PostgreSQL connection

```
Connection connection = DriverManager.getConnection(
    "jdbc:postgresql://localhost:5432/student_db",
    "postgres",
    "password"
);
```

This provides portability at the API level. It does not mean that every database supports identical SQL syntax or behaviour.

JDBC API versus JDBC driver

JDBC API

The JDBC API defines the contracts used by Java code.

Important interfaces include:

`Connection`
`Statement`
`PreparedStatement`
`ResultSet`

For example, `Connection` defines operations such as creating statements, managing transactions and closing a database session.

Your application normally programs against these interfaces rather than database-specific implementation classes.

JDBC driver

The JDBC driver provides the database-specific implementation of the JDBC contracts.

For MySQL, the official driver is **MySQL Connector/J**.

Database	Common JDBC driver
MySQL	MySQL Connector/J
PostgreSQL	PostgreSQL JDBC Driver
Oracle Database	Oracle JDBC Driver
Microsoft SQL Server	Microsoft JDBC Driver for SQL Server
H2	H2 JDBC Driver

What happens internally?

When Java executes a query through JDBC, the flow is approximately:

1. The Java application calls a JDBC method.
2. The JDBC driver receives the SQL command and parameter values.
3. The driver converts them into a database-specific network request.

4. The request is sent to the database server.
 5. The server parses and validates the SQL.
 6. The server creates an execution plan.
 7. The server executes the operation.
 8. The server returns rows, metadata, an update count or an error.
 9. The driver decodes the database response.
 10. JDBC exposes the response through types such as `ResultSet`.
 11. The Java application reads and maps the returned data.
-

3. Installing MySQL Server and DBeaver

MySQL Server, DBeaver and the JDBC driver are different

Component	Purpose
MySQL Server	Stores data and executes SQL
DBeaver	Desktop client used to view databases and run SQL manually
MySQL Connector/J	Java library that allows the application to communicate with MySQL

Installing DBeaver does **not** install or start MySQL Server. The server must be installed and running separately.

3.1 Install MySQL Server on macOS

Option A: Official MySQL package installer

1. Open the official [MySQL Community Server download page](#).
2. Select **macOS**.
3. Download the DMG package that matches your Mac:
 - Apple Silicon for M-series Macs

- x86-64 for Intel Macs
4. Open the downloaded `.dmg` file.
 5. Run the MySQL `.pkg` installer.
 6. Complete the installation wizard.
 7. Set and safely store the MySQL `root` password when prompted.

The official package normally installs MySQL under:

```
/usr/local/mysql
```

Start MySQL from macOS settings

Depending on the installer and macOS version:

1. Open **System Settings**.
2. Search for **MySQL**.
3. Open the MySQL settings panel.
4. Select **Start MySQL Server**.

Start MySQL from Terminal

```
sudo /usr/local/mysql/support-files/mysql.server start
```

Check its status:

```
sudo /usr/local/mysql/support-files/mysql.server status
```

Stop the server:

```
sudo /usr/local/mysql/support-files/mysql.server stop
```

Restart the server:

```
sudo /usr/local/mysql/support-files/mysql.server restart
```

Connect through the MySQL command-line client

```
/usr/local/mysql/bin/mysql -u root -p
```

To make the `mysql` command available without its full path, add MySQL to the shell path:

```
echo 'export PATH="/usr/local/mysql/bin:$PATH"' >> ~/.zshrc  
source ~/.zshrc
```

You can then run:

```
mysql -u root -p
```

Option B: Install MySQL with Homebrew

This option is convenient for developers who already use Homebrew.

```
brew install mysql
```

Start MySQL and configure it to launch automatically:

```
brew services start mysql
```

Check the service:

```
brew services list
```

Stop it:

```
brew services stop mysql
```


Restart it:

```
brew services restart mysql
```

Run the security setup:

```
mysql_secure_installation
```

Connect to the server:

```
mysql -u root -p
```

Use either the official package installation or Homebrew. Avoid maintaining two separate MySQL installations unless you understand how their service paths and ports differ.

3.2 Install MySQL Server on Windows

1. Open the official [MySQL Community Server download page](#).
2. Download the Windows MSI package.
3. Run the installer as an administrator.
4. Open the MySQL Configurator when prompted.
5. Configure MySQL as a standalone server.
6. Keep port `3306` unless another program already uses it.
7. Set and safely store the `root` password.
8. Configure MySQL as a **Windows service**.
9. Keep **Start the MySQL Server at System Startup** selected for a beginner setup.
10. Apply the configuration.

Start MySQL using Windows Services

1. Press `Win + R`.

2. Enter:

```
services.msc
```

1. Find the MySQL service. Its name may be similar to `MySQL80` or `MySQL84`.

2. Right-click it and select **Start**, **Stop** or **Restart**.

Start MySQL from Command Prompt

Open Command Prompt as administrator.

First, identify the actual service name:

```
sc query type= service state= all | findstr /I mysql
```

Start the service:

```
net start <service-name>
```

Example:

```
net start MySQL84
```

Stop the service:

```
net stop <service-name>
```

You may also use:

```
sc start <service-name>
sc stop <service-name>
```

PowerShell alternative

Open PowerShell as administrator:

```
Get-Service *mysql*
Start-Service -Name <service-name>
Stop-Service -Name <service-name>
Restart-Service -Name <service-name>
```

Connect through the command-line client

If the MySQL `bin` directory is available in `PATH`:

```
mysql -u root -p
```

Otherwise, run it from the installation directory, which is commonly similar to:

```
C:\Program Files\MySQL\MySQL Server 8.4\bin
```

The exact directory depends on the installed version.

3.3 Verify that MySQL is running

Log in:

```
mysql -u root -p
```

Then run:

```
SELECT VERSION();
SHOW DATABASES;
```

Exit the MySQL client:

```
EXIT;
```

If the command-line login works, the server is running and accepting connections.

3.4 Install DBeaver Community

Download DBeaver Community from the official [DBeaver download page](#).

macOS

1. Download the correct DMG for Apple Silicon or Intel.
2. Open the DMG.
3. Drag DBeaver into the **Applications** folder.
4. Open DBeaver from Applications.

Homebrew alternative:

```
brew install --cask dbeaver-community
```

Windows

1. Download the Windows installer.
2. Run the installer executable.
3. Follow the installation wizard.
4. Launch DBeaver.

3.5 Connect DBeaver to MySQL

1. Make sure MySQL Server is running.
2. Open DBeaver.
3. Select **New Database Connection**.
4. Choose **MySQL**.
5. Enter the connection details:

Field	Value
Host	localhost
Port	3306

Field	Value
Database	<code>student_db</code> or leave empty initially
Username	<code>root</code>
Password	Your MySQL root password

1. Allow DBeaver to download the MySQL driver if it asks.
2. Select **Test Connection**.
3. Select **Finish** after the test succeeds.

4. Creating the JDBC Project

Recommended project structure

```

jdbc-crud-demo
├── pom.xml
├── src
│   └── main
│       ├── java
│       │   ├── com
│       │   │   └── coderarmy
│       │   │       ├── jdbc
│       │   │       │   ├── Main.java
│       │   │       │   ├── model
│       │   │       │   │   └── Student.java
│       │   │       │   └── repository
│       │   │       │       └── StudentRepository.java

```

This is a plain Maven project. Spring Boot is not required to learn raw JDBC.

Add MySQL Connector/J

Add the MySQL JDBC driver to `pom.xml`:

```

<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>

```

```
<version>9.7.0</version>
<scope>runtime</scope>
</dependency>
```

Dependency versions change over time. Use the current stable Connector/J version when creating a new project.

A minimal `pom.xml` can look like this:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.coderarmy</groupId>
  <artifactId>jdbc-crud-demo</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.release>21</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <version>9.7.0</version>
      <scope>runtime</scope>
    </dependency>
```

```
</dependencies>

</project>
```

5. Creating the Database and Table

Open an SQL editor in DBeaver or use the MySQL command-line client.

Create the database

```
CREATE DATABASE student_db;
```

Select it:

```
USE student_db;
```

Create the **students** table

```
CREATE TABLE students (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  age INT
);
```

Verify the table:

```
DESCRIBE students;
```

Create the **Student** model

```
package com.coderarmy.jdbc.model;

public class Student {

    private Long id;
    private String name;
    private String email;
    private Integer age;

    public Student() {
    }

    public Student(String name, String email, Integer age) {
        this.name = name;
        this.email = email;
        this.age = age;
    }

    public Student(Long id, String name, String email, Integer age) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.age = age;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
```



```
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public Integer getAge() {
        return age;
    }

    public void setAge(Integer age) {
        this.age = age;
    }

    @Override
    public String toString() {
        return "Student{" +
            "id=" + id +
            ", name='" + name + '\'' +
            ", email='" + email + '\'' +
            ", age=" + age +
            '}';
    }
}
```

ction

ssion between the

What is a database connection?

A database connection represents an active communication session between the Java application and the database server.

In JDBC, it is represented by the `Connection` interface:

```
Connection connection;
```

A connection allows Java to:

- Send SQL commands
- Receive query results
- Create statements
- Start and commit transactions
- Roll back transactions
- Read database metadata

Opening a database connection involves network communication and authentication. It is more expensive than creating an ordinary Java object.

JDBC connection details

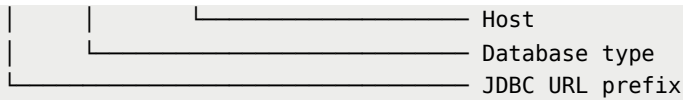
```
String url = "jdbc:mysql://localhost:3306/student_db";
String username = "root";
String password = "your password";
```

URL breakdown

jdbc:mysql://localhost:3306/student_db

Database name

Port



For learning, these values may be kept as constants.

In production code, credentials should not be committed to source control. Use environment variables, configuration files excluded from Git or a secret-management system.

`DriverManager` does not automatically read Spring Boot's `application.properties`. Values must be passed directly unless Spring is creating and configuring a `DataSource`.

Is `Class.forName()` still required?

Older JDBC applications explicitly loaded the MySQL driver:

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

This forced the driver class to load and register itself with `DriverManager`.

Since JDBC 4.0, compatible drivers can be discovered and registered automatically when the driver JAR is present on the classpath.

Therefore, modern JDBC applications generally do not need `Class.forName()`.

How `DriverManager` creates a connection

```
Connection connection = DriverManager.getConnection(  
    url,  
    username,  
    password  
);
```

Conceptually, `DriverManager`:

1. Looks at the JDBC URL.

2. Checks the registered JDBC drivers.
3. Finds the driver that supports `jdbc:mysql:`.
4. Asks that driver to connect to MySQL.
5. Sends the authentication details.
6. Returns a MySQL-specific implementation of `Connection` through the JDBC interface.

Your code depends on:

`Connection`

rather than a MySQL implementation class. This is interface-based abstraction in practice.

Basic connection program

```
package com.coderarmy.jdbc;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class Main {

    private static final String URL =
        "jdbc:mysql://localhost:3306/student_db";

    private static final String USERNAME = "root";
    private static final String PASSWORD = "your_password";

    public static void main(String[] args) {

        try (Connection connection = DriverManager.getConnection(
```

```

        URL,
        USERNAME,
        PASSWORD
    )) {
        System.out.println("Database connected successful
ly");

    } catch (SQLException exception) {
        System.out.println("Database connection failed");
        System.out.println("Message: " + exception.getMes
sage());
        System.out.println("SQL state: " + exception.getSQLState());
        System.out.println("Error code: " + exception.getErrorCode());
    }
}
}

```

Useful information available from `SQLException`:

```

exception.getMessage();
exception.getSQLState();
exception.getErrorCode();

```

7. Executing SQL with JDBC

From a connection to a statement

A `Connection` represents the database session. It does not represent one specific SQL command.

A statement object is created to execute SQL through that connection.

```
Connection connection = DriverManager.getConnection(
    URL,
    USERNAME,
    PASSWORD
);

Statement statement = connection.createStatement();
```

```
Connection
└─ Statement
```

The statement uses its parent connection to communicate with MySQL.

Main JDBC execution methods

Method	Typical use	Return value
<code>executeQuery()</code>	<code>SELECT</code>	<code>ResultSet</code>
<code>executeUpdate()</code>	<code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code> , some DDL	Affected-row count
<code>execute()</code>	Unknown or mixed result type	<code>boolean</code> indicating the first result type

`executeUpdate()`

Despite its name, `executeUpdate()` is used for more than SQL `UPDATE` .

Typical uses:

```
INSERT
UPDATE
DELETE
```

Example:

```
int rowsAffected = statement.executeUpdate(
    """
    INSERT INTO students(name, email, age)
    VALUES ('Rahul', 'rahul@gmail.com', 24)
    """
);
```

The returned integer represents the number of affected rows.

```
Successful single-row insert → 1
No row matched an update    → 0
Multiple rows affected      → more than 1
```

It can also execute DDL such as:

```
CREATE TABLE
ALTER TABLE
DROP TABLE
```

DDL commonly returns `0` because it does not modify table rows in the same way as DML.

executeQuery()

`executeQuery()` is used when SQL returns tabular data, normally with `SELECT`.

```
ResultSet resultSet = statement.executeQuery(
    "SELECT id, name, email, age FROM students"
);
```

The returned `ResultSet` gives Java access to the rows returned by MySQL.

JDBC does not automatically convert these rows into `Student` objects. The application must map them manually.

`execute()`

`execute()` is a general-purpose method:

```
boolean hasResultSet = statement.execute(sql);
```

The boolean does **not** mean success or failure.

```
true  → The first result is a ResultSet
false → The first result is an update count or there is no result
```

Example:

```
boolean hasResultSet = statement.execute(sql);

if (hasResultSet) {
    ResultSet resultSet = statement.getResultSet();
} else {
    int rowsAffected = statement.getUpdateCount();
}
```

Use `execute()` when:

- The SQL type is not known in advance
- A stored procedure can return different result types
- Multiple results may be returned
- Infrastructure code needs a generic execution method

For normal CRUD code, prefer the more specific methods:

```
SELECT          → executeQuery()
INSERT/UPDATE/DELETE → executeUpdate()
```

8. Understanding `ResultSet`

ResultSet is not a List<Student>

Suppose MySQL returns:

id	name	email	age
1	Rahul	rahul@gmail.com	24
2	Priya	priya@gmail.com	22

JDBC exposes these rows through `ResultSet`.

The developer must:

1. Move the cursor to a row.
2. Read each required column.
3. Create a Java object.
4. Assign the values.
5. Add the object to a collection when reading multiple rows.

The `ResultSet` cursor

When a `ResultSet` is created, its cursor is positioned **before the first row**.

Cursor → before first row

Row 1
Row 2
Row 3

Calling:

```
resultSet.next();
```

moves the cursor to the next row and returns:

- `true` when a row is available
- `false` when no more rows remain

For one expected row:

```
if (resultSet.next()) {  
    // Read the current row  
}
```

For multiple rows:

```
while (resultSet.next()) {  
    // Read each row  
}
```

Reading values

By column label

```
long id = resultSet.getLong("id");  
String name = resultSet.getString("name");  
String email = resultSet.getString("email");  
int age = resultSet.getInt("age");
```

By column position

```
long id = resultSet.getLong(1);  
String name = resultSet.getString(2);  
String email = resultSet.getString(3);  
int age = resultSet.getInt(4);
```

Column labels are usually clearer because the code states which value it is reading.

Handling nullable numeric columns

Calling:

```
resultSet.getInt("age");
```

returns primitive `int`. If the SQL value is `NULL`, JDBC returns `0`, and the code must call `wasNull()` to distinguish SQL `NULL` from a real zero.

A clearer alternative for a nullable `Integer` field is:

```
Integer age = resultSet.getObject("age", Integer.class);
```

Prefer explicit columns over `SELECT *`

Instead of:

```
SELECT *  
FROM students;
```

prefer:

```
SELECT id, name, email, age  
FROM students;
```

Benefits:

- The required columns are visible
- Unnecessary columns are not retrieved
- Mapping is more predictable
- Schema changes are less likely to surprise the application
- The repository clearly communicates the data it needs

`SELECT *` is convenient during exploration, but explicit columns are better in application code.

Mapping one row to a `Student`

```
private Student mapRow(ResultSet resultSet) throws SQLException {
    Student student = new Student();

    student.setId(resultSet.getLong("id"));
    student.setName(resultSet.getString("name"));
    student.setEmail(resultSet.getString("email"));
    student.setAge(resultSet.getObject("age", Integer.class));

    return student;
}
```

This method removes duplicate mapping code from repository methods.

The mapping is still manual. JDBC does not know that a row from `students` should become a `Student` object.

9. Why `PreparedStatement` Is Preferred

The problem with plain `Statement`

A hardcoded statement is simple:

```
String sql = """
    INSERT INTO students(name, email, age)
    VALUES ('Rahul', 'rahul@gmail.com', 24)
    """;
```

Real applications receive dynamic values:

```
String name = student.getName();
String email = student.getEmail();
```

```
Integer age = student.getAge();
```

Building SQL through concatenation is unsafe and error-prone:

```
String sql =  
    "INSERT INTO students(name, email, age) VALUES ("  
        + name + "', '"  
        + email + "', "  
        + age + ")";
```

Formatting a text block does not solve the underlying problem:

```
String sql = ""  
    INSERT INTO students(name, email, age)  
    VALUES ('%s', '%s', %d)  
    """.formatted(name, email, age);
```

Problems include:

- SQL injection
- Broken quotation when text contains apostrophes
- Incorrect handling of `NULL`
- Manual type formatting
- Date and time conversion issues
- Binary-data handling issues
- Poor readability and maintainability

Parameterized SQL

`PreparedStatement` separates the SQL structure from its values.

```
String sql = ""  
    INSERT INTO students(name, email, age)  
    VALUES (?, ?, ?)
```

```

        """;

PreparedStatement preparedStatement =
    connection.prepareStatement(sql);

preparedStatement.setString(1, student.getName());
preparedStatement.setString(2, student.getEmail());
preparedStatement.setObject(3, student.getAge());

int rowsAffected = preparedStatement.executeUpdate();

```

The SQL is sent as a parameterized command, while the values are supplied separately.

Positional parameters

In this query:

```

INSERT INTO students(name, email, age)
VALUES (?, ?, ?)

```

the placeholders mean:

```

First ?   → name
Second ?  → email
Third ?   → age

```

JDBC parameter indexes begin at **1**, not **0**.

Common setter methods

```

setString()
setInt()
setLong()
setDouble()

```

```
setBoolean()
setBigDecimal()
setDate()
setTimestamp()
setObject()
setNull()
```

For a nullable value:

```
if (student.getAge() == null) {
    preparedStatement.setNull(3, java.sql.Types.INTEGER);
} else {
    preparedStatement.setInt(3, student.getAge());
}
```

A simpler alternative is often:

```
preparedStatement.setObject(3, student.getAge());
```

Does **PreparedStatement** completely prevent SQL injection?

It safely handles values supplied through placeholders:

```
String sql = "SELECT id, name, email, age FROM students WHERE email = ?";
preparedStatement.setString(1, email);
```

However, placeholders cannot normally represent SQL identifiers or keywords.

This remains unsafe:

```
String sql = "SELECT * FROM students ORDER BY " + requestedColumn;
```

For dynamic table names, column names or sort directions, validate the input against a strict allow-list.

Use placeholders for values. Use allow-list validation for unavoidable dynamic SQL structure.

10. Complete CRUD Operations

Repository configuration

```
package com.coderarmy.jdbc.repository;

import com.coderarmy.jdbc.model.Student;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public class StudentRepository {

    private static final String URL =
        "jdbc:mysql://localhost:3306/student_db";

    private static final String USERNAME = "root";
    private static final String PASSWORD = "your_password";

    // CRUD methods
}
```


For a real application, move credentials outside the source code and use a connection pool. They remain inline here only to keep the first JDBC example focused.

10.1 Create: insert a student

```
public int save(Student student) {

    String sql = ""
        INSERT INTO students(name, email, age)
        VALUES (?, ?, ?)
        "";

    try (
        Connection connection = DriverManager.getConnection(
            URL,
            USERNAME,
            PASSWORD
        );

        PreparedStatement preparedStatement =
            connection.prepareStatement(sql)
    ) {
        preparedStatement.setString(1, student.getName());
        preparedStatement.setString(2, student.getEmail());
        preparedStatement.setObject(3, student.getAge());

        return preparedStatement.executeUpdate();

    } catch (SQLException exception) {
        throw new RuntimeException(
            "Failed to save student",
            exception
        );
    }
}
```

```
}  
}
```

Usage:

```
Student student = new Student(  
    "Rahul",  
    "rahul@gmail.com",  
    24  
);  
  
int rowsAffected = repository.save(student);  
  
if (rowsAffected == 1) {  
    System.out.println("Student saved successfully");  
}
```

Returning the generated ID

To retrieve an auto-generated key:

```
public long saveAndReturnId(Student student) {  
  
    String sql = ""  
        INSERT INTO students(name, email, age)  
        VALUES (?, ?, ?)  
        "";  
  
    try (  
        Connection connection = DriverManager.getConnection(  
            URL,  
            USERNAME,  
            PASSWORD  
        );  
    }
```

```

        PreparedStatement preparedStatement =
            connection.prepareStatement(
                sql,
                Statement.RETURN_GENERATED_KEYS
            )
    ) {
        preparedStatement.setString(1, student.getName());
        preparedStatement.setString(2, student.getEmail());
        preparedStatement.setObject(3, student.getAge());

        int rowsAffected = preparedStatement.executeUpdate();

        if (rowsAffected != 1) {
            throw new RuntimeException("Student was not inserted");
        }

        try (ResultSet generatedKeys = preparedStatement.getGeneratedKeys()) {
            if (generatedKeys.next()) {
                return generatedKeys.getLong(1);
            }
        }

        throw new RuntimeException("Generated ID was not returned");
    } catch (SQLException exception) {
        throw new RuntimeException(
            "Failed to save student",
            exception
        );
    }
}

```

10.2 Read one: find a student by ID

```
public Optional<Student> findById(Long id) {

    String sql = """
        SELECT id, name, email, age
        FROM students
        WHERE id = ?
        """;

    try (
        Connection connection = DriverManager.getConnection(
            URL,
            USERNAME,
            PASSWORD
        );

        PreparedStatement preparedStatement =
            connection.prepareStatement(sql)
    ) {
        preparedStatement.setLong(1, id);

        try (ResultSet resultSet = preparedStatement.executeQuery()) {
            if (resultSet.next()) {
                return Optional.of(mapRow(resultSet));
            }

            return Optional.empty();
        }
    } catch (SQLException exception) {
        throw new RuntimeException(
            "Failed to fetch student with ID " + id,
            exception
        );
    }
}
```

```
    );
}
}
```

Usage:

```
Optional<Student> optionalStudent = repository.findById(1L);

if (optionalStudent.isPresent()) {
    System.out.println(optionalStudent.get());
} else {
    System.out.println("Student not found");
}
```

A more concise usage:

```
repository.findById(1L)
    .ifPresentOrElse(
        System.out::println,
        () -> System.out.println("Student not found")
    );
```

`Optional.empty()` represents a normal “student not found” result. A database failure is different and should not be silently converted into `Optional.empty()`.

10.3 Read all: fetch every student

```
public List<Student> findAll() {

    String sql = """
        SELECT id, name, email, age
        FROM students
        """;

    List<Student> students = new ArrayList<>();
```

```
try (
    Connection connection = DriverManager.getConnection(
        URL,
        USERNAME,
        PASSWORD
    );

    PreparedStatement preparedStatement =
        connection.prepareStatement(sql);

    ResultSet resultSet =
        preparedStatement.executeQuery()
) {
    while (resultSet.next()) {
        students.add(mapRow(resultSet));
    }

    return students;

} catch (SQLException exception) {
    throw new RuntimeException(
        "Failed to fetch students",
        exception
    );
}
```

Flow:

```
Move to the next row
↓
Read the columns
↓
Create a Student
↓
Add it to the List
```

↓
Repeat until next() returns false

10.4 Update a student

```
public int update(Long id, Student student) {

    String sql = ""
        UPDATE students
        SET name = ?,
            email = ?,
            age = ?
        WHERE id = ?
        "";

    try (
        Connection connection = DriverManager.getConnection(
            URL,
            USERNAME,
            PASSWORD
        );

        PreparedStatement preparedStatement =
            connection.prepareStatement(sql)
    ) {
        preparedStatement.setString(1, student.getName());
        preparedStatement.setString(2, student.getEmail());
        preparedStatement.setObject(3, student.getAge());
        preparedStatement.setLong(4, id);

        return preparedStatement.executeUpdate();

    } catch (SQLException exception) {
        throw new RuntimeException(
```

```

        "Failed to update student with ID " + id,
        exception
    );
}
}

```

Usage:

```

Student updatedStudent = new Student(
    "Rahul Sharma",
    "rahul.sharma@gmail.com",
    25
);

int rowsAffected = repository.update(1L, updatedStudent);

if (rowsAffected == 1) {
    System.out.println("Student updated successfully");
} else {
    System.out.println("Student not found");
}

```

Understanding the update count

For a simple beginner-level example:

```

1 → one row was matched or changed
0 → no row was matched, or the driver reported no changed row

```

Exact update-count semantics can vary by database and driver configuration. MySQL Connector/J settings such as `useAffectedRows` can influence whether the count reflects matched rows or changed rows.

Do not treat update count as the only source of truth for critical business rules without understanding the driver behaviour.

Should `findById()` be called before `update()` ?

This approach performs two SQL operations:

```
SELECT student
    ↓
UPDATE student
```

Problems:

- An extra database round trip
- Additional latency
- A possible race condition

The row can be deleted or changed by another transaction after the `SELECT` and before the `UPDATE`.

For a basic CRUD operation, execute the `UPDATE` directly and inspect its update count.

Perform an additional query only when the application genuinely needs more information, such as distinguishing between:

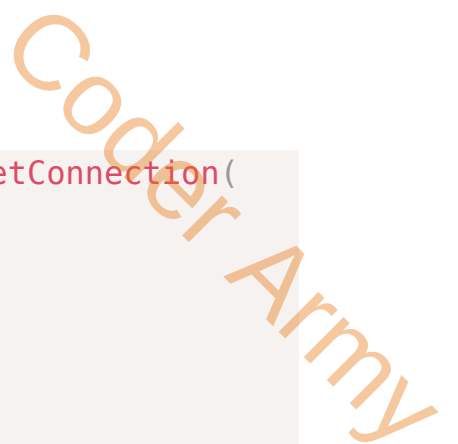
- Row not found
- Row found but values unchanged
- Optimistic-locking conflict

10.5 Delete a student

```
public int deleteById(Long id) {

    String sql = """
        DELETE FROM students
        WHERE id = ?
        """;

    try (
```



```

        Connection connection = DriverManager.getConnection(
            URL,
            USERNAME,
            PASSWORD
        );

        PreparedStatement preparedStatement =
            connection.prepareStatement(sql)
    ) {
        preparedStatement.setLong(1, id);

        return preparedStatement.executeUpdate();

    } catch (SQLException exception) {
        throw new RuntimeException(
            "Failed to delete student with ID " + id,
            exception
        );
    }
}

```

Usage:

```

int rowsAffected = repository.deleteById(1L);

if (rowsAffected == 1) {
    System.out.println("Student deleted successfully");
} else {
    System.out.println("Student not found");
}

```

For a delete operation:

```

1 → one row deleted
0 → no matching row found

```

Complete `mapRow()` method

```
private Student mapRow(ResultSet resultSet) throws SQLException {
    return new Student(
        resultSet.getLong("id"),
        resultSet.getString("name"),
        resultSet.getString("email"),
        resultSet.getObject("age", Integer.class)
    );
}
```

11. Closing JDBC Resources

Resource dependency

JDBC resources have a parent-child relationship:

```
Connection
  ↓ creates
PreparedStatement
  ↓ creates
ResultSet
```

A `ResultSet` depends on the statement that produced it. The statement depends on the connection used to execute it.

Resources should therefore be closed in reverse order:

```
ResultSet
  ↓
PreparedStatement
  ↓
Connection
```

Manual closing with **finally**

Before try-with-resources, JDBC code commonly followed this structure:

```
Connection connection = null;
PreparedStatement preparedStatement = null;
ResultSet resultSet = null;

try {
    connection = DriverManager.getConnection(
        URL,
        USERNAME,
        PASSWORD
    );

    preparedStatement = connection.prepareStatement(sql);
    resultSet = preparedStatement.executeQuery();

    // Process data

} catch (SQLException exception) {
    throw new RuntimeException(exception);
} finally {
    try {
        if (resultSet != null) {
            resultSet.close();
        }
    } catch (SQLException exception) {
        exception.printStackTrace();
    }

    try {
        if (preparedStatement != null) {
            preparedStatement.close();
        }
    } catch (SQLException exception) {
        exception.printStackTrace();
    }
}
```

```
    }  
    } catch (SQLException exception) {  
        exception.printStackTrace();  
    }  
  
    try {  
        if (connection != null) {  
            connection.close();  
        }  
    } catch (SQLException exception) {  
        exception.printStackTrace();  
    }  
}
```

This works, but it is verbose and easy to get wrong.

Try-with-resources

`Connection`, `Statement`, `PreparedStatement` and `ResultSet` implement `AutoCloseable`.

They can be declared inside try-with-resources:

```
try (  
    Connection connection = DriverManager.getConnection(  
        URL,  
        USERNAME,  
        PASSWORD  
    );  
  
    PreparedStatement preparedStatement =  
        connection.prepareStatement(sql);  
  
    ResultSet resultSet =  
        preparedStatement.executeQuery()  
) {
```

```
// Use the resources  
}
```

Java closes them automatically in reverse declaration order:

```
ResultSet → PreparedStatement → Connection
```

Try-with-resources is the recommended approach for normal JDBC code.

Why closing the connection matters

A database connection holds resources in both the application and the database server.

Failing to close connections can cause:

- Resource leaks
- Too many open database sessions
- Application slowdown
- "Too many connections" errors
- Uncommitted transactions remaining open

In this basic project, `connection.close()` closes the physical connection.

In applications that use a connection pool, `close()` usually returns the connection to the pool instead of shutting down the underlying network connection.

12. Common Setup Problems

Connection refused

Typical error:

```
Communications link failure  
Connection refused
```

Check:

1. Is MySQL Server running?
2. Is the host correct?
3. Is the port `3306` correct?
4. Is a firewall blocking the connection?
5. Is another MySQL installation using a different port?

Access denied

Typical error:

```
Access denied for user 'root'@'localhost'
```

Check:

- Username
- Password
- MySQL user permissions
- Host from which the user is allowed to connect

Unknown database

Typical error:

```
Unknown database 'student_db'
```

Create the database first:

```
CREATE DATABASE student_db;
```

MySQL command not found on macOS

For the official package installation:

```
/usr/local/mysql/bin/mysql -u root -p
```

Or add it to the shell path:

```
echo 'export PATH="/usr/local/mysql/bin:$PATH"' >> ~/.zshrc  
source ~/.zshrc
```

MySQL service name not found on Windows

List services containing "mysql":

```
sc query type= service state= all | findstr /I mysql
```

Then use the exact returned service name:

```
net start <service-name>
```

DBeaver: "Public Key Retrieval is not allowed"

This error can occur with some local MySQL authentication configurations.

For a local development connection in DBeaver:

1. Edit the connection.
2. Open **Driver Properties**.
3. Set:

```
allowPublicKeyRetrieval = true
```

If the local server is not configured for TLS, some learners also use:


```
useSSL = false
```

Use `useSSL=false` only for a trusted local development environment. Remote and production database connections should use properly configured TLS.

Duplicate email error

The table defines:

```
email VARCHAR(150) NOT NULL UNIQUE
```

Inserting an existing email produces a constraint violation. This is expected database behaviour and should eventually be handled with application-specific exception handling.

13. Key Takeaways

1. A Java application and a database server are separate programs.
2. JDBC is the standard Java API for relational database communication.
3. A JDBC driver implements the database-specific protocol.
4. `Connection` represents an active database session.
5. `Statement` executes plain SQL, while `PreparedStatement` executes parameterized SQL.
6. Use `executeQuery()` for queries that return rows.
7. Use `executeUpdate()` for insert, update and delete operations.
8. The boolean returned by `execute()` identifies the first result type, not success or failure.
9. A `ResultSet` cursor starts before the first row.
10. JDBC rows must be mapped manually into Java objects.
11. Prefer explicit column lists over `SELECT *`.

12. Use `PreparedStatement` for dynamic values.
13. Placeholders protect values, not dynamic SQL identifiers.
14. Close `ResultSet`, statement and connection resources.
15. Prefer try-with-resources over manual cleanup in `finally`.
16. MySQL Server must be running before Java or DBeaver can connect to it.
17. DBeaver is a client; it does not replace the MySQL server.
18. Raw JDBC is repetitive by design, which helps explain the value later provided by connection pools, `DataSource`, Spring JDBC, JPA and Hibernate.
-

Complete communication flow

```
graph TD; A[Java Application] --> B[JDBC Interfaces]; B --> C[MySQL Connector/J]; C --> D[Network Connection]; D --> E[MySQL Server]; E --> F[SQL Execution]; F --> G[Rows / Update Count / Error]; G --> H[MySQL Connector/J]; H --> I[ResultSet or affected-row count]; I --> J[Java object mapping];
```

Java Application

↓

JDBC Interfaces

↓

MySQL Connector/J

↓

Network Connection

↓

MySQL Server

↓

SQL Execution

↓

Rows / Update Count / Error

↓

MySQL Connector/J

↓

ResultSet or affected-row count

↓

Java object mapping